Python Tkinter GUI

Beginner's Complete Guide

From Zero to SQL-Connected Desktop Apps

❖ Widgets ❖ Layouts ❖ Events ❖ Forms ❖ SQLite DB ❖

This guide teaches Python Tkinter GUI programming from absolute zero. Every topic is explained in plain language with real, runnable code examples and step-by-step explanations. By the end you will build a fully working Student Management System backed by an SQLite database.

Who is this for?	
✔ Complete beginners	No prior GUI experience needed
✔ Python learners	Basic Python syntax is enough
✔ Students / Teachers	Great for classroom use
✔ Hobbyists	Build real desktop apps fast

■ Table of Contents

Chapter 1: Introduction to Tkinter

- ■ What is Tkinter?
- ■ Why use it?
- ■ Your first window

Chapter 2: Basic Widgets

- ■ Labels
- ■ Buttons
- ■ Entry (text input)
- ■ Text widget
- ■ Checkbutton & Radiobutton

Chapter 3: Layout Managers

- ■ pack()
- ■ grid()
- ■ place()
- ■ When to use which

Chapter 4: Event Handling

- ■ Command callbacks
- ■ bind() method
- ■ Keyboard & mouse events

Chapter 5: More Widgets

- ■ Listbox
- ■ Combobox
- ■ Spinbox
- ■ Scale (slider)
- ■ MessageBox dialogs

Chapter 6: Frames & Organisation

- ■ Frame widget
- ■ LabelFrame
- ■ Notebook (tabs)

Chapter 7: Forms & Validation

- ■ Building a form
- ■ Input validation
- ■ Showing errors

Chapter 8: SQLite & Tkinter

- ■ What is SQLite?
- ■ sqlite3 module
- ■ Connecting to a DB

Chapter 9: CRUD Operations with GUI

- ■ Create
- ■ Read (Treeview)
- ■ Update
- ■ Delete

Chapter 10: Full Project: Student Management System

- ■ Complete working application with all CRUD ops

Chapter 11: Tips, Best Practices & Next Steps

- Common mistakes
- Packaging
- What to learn next

1 Introduction to Tkinter

1.1 What is Tkinter?

Tkinter is Python's **built-in library** for creating Graphical User Interfaces (GUIs). A GUI is any program that has windows, buttons, and input boxes — like Notepad or Calculator on Windows.

Tkinter is short for **Tk interface**. It wraps a toolkit called Tk (originally from the Tcl language) and gives us a clean Python API to use it.

1.2 Why Tkinter?

■ Built-in	No installation needed — comes with Python
■ Simple	Easy to learn, great for beginners
■ Cross-platform	Works on Windows, Mac, and Linux
■ Enough features	Buttons, tables, menus, dialogs — all included

1.3 Setting Up

Tkinter comes with Python. Just check your Python version:

```
python --version   # should be 3.x
```

If Tkinter is missing (rare on Linux), install it:

```
sudo apt-get install python3-tk   # Ubuntu/Debian
```

1.4 Your Very First Window

Let's write the simplest possible Tkinter program:

```
import tkinter as tk          # Step 1: import tkinter

window = tk.Tk()              # Step 2: create the main window
window.title("Hello World!")  # Step 3: set the window title
window.geometry("400x300")    # Step 4: set width x height in pixels

window.mainloop()             # Step 5: start the event loop (keep window open)
```

■ **Run this** — a blank 400×300 window titled 'Hello World!' appears.

Line	What it does
import tkinter as tk	Loads the Tkinter library. We nickname it 'tk' to save typing.
tk.Tk()	Creates the main application window. Every app has exactly one.

window.title(...)	Sets the text shown in the window's title bar.
window.geometry('400x300')	Sets the window size: 400 pixels wide, 300 pixels tall.
window.mainloop()	Starts an infinite loop that waits for user actions (clicks, typing). Without this, the window

■ ***Tip:*** *Always put window.mainloop() as the LAST line. Nothing after it will run until the window is closed.*

2 Basic Widgets

A **widget** is any element you put inside a window — a button, a label, a text box, etc. Think of widgets as the building blocks of your GUI.

2.1 Label — Displaying Text

A **Label** simply shows text (or an image) on the screen. The user cannot type into it.

```
import tkinter as tk

window = tk.Tk()
window.title("Label Demo")

# Create a label widget
lbl = tk.Label(window, text="Hello, I am a Label!")

# Place it in the window using pack()
lbl.pack()

window.mainloop()
```

You can style a label with extra options:

```
lbl = tk.Label(
    window,
    text    = "Styled Label",
    font    = ("Arial", 16, "bold"),   # font family, size, style
    fg      = "blue",                  # text colour
    bg      = "lightyellow",           # background colour
    padx    = 10,                      # horizontal padding (pixels)
    pady    = 10                       # vertical padding (pixels)
)
lbl.pack()
```

2.2 Button — Clickable Actions

A **Button** runs a function when clicked. You connect it to a function using the **command** option.

```
import tkinter as tk

def say_hello():                        # This function runs on button click
```

```
    print("Button was clicked!")

window = tk.Tk()
window.title("Button Demo")

btn = tk.Button(
    window,
    text    = "Click Me!",
    command = say_hello,              # <-- link to function (no brackets!)
    bg      = "#4caf50",              # green background
    fg      = "white",
    font    = ("Arial", 12),
    width   = 15,
    height  = 2
)
btn.pack(pady=20)

window.mainloop()
```

■ ***Tip:*** *Pass the function NAME (say_hello) not a call (say_hello()). With brackets, it runs immediately at startup instead of on click.*

2.3 Entry — Single-Line Text Input

An **Entry** widget lets the user type a single line of text. Use **.get()** to read what they typed.

```
import tkinter as tk

def greet():
    name = entry.get()                 # read text from Entry
    label.config(text=f"Hello, {name}!")

window = tk.Tk()
window.title("Entry Demo")

tk.Label(window, text="Enter your name:").pack()

entry = tk.Entry(window, width=30)
entry.pack(pady=5)

tk.Button(window, text="Greet", command=greet).pack()
```

```
label = tk.Label(window, text="", fg="green", font=("Arial",12))
label.pack(pady=10)

window.mainloop()
```

Useful Entry methods:

Method	What it does
`entry.get()`	Returns the current text as a string
`entry.delete(0, tk.END)`	Clears all text from the entry
`entry.insert(0, 'text')`	Inserts text at position 0 (the start)
`entry.config(show='*')`	Hides characters (use for passwords)

2.4 Text — Multi-Line Input

Use **Text** when you need a big box for multiple lines (like a notes area).

```
import tkinter as tk

window = tk.Tk()
window.title("Text Widget Demo")

txt = tk.Text(window, width=40, height=8, font=("Arial", 11))
txt.pack(padx=10, pady=10)

def read_text():
    content = txt.get("1.0", tk.END)  # "1.0" = line 1, char 0
    print(content)

tk.Button(window, text="Read Text", command=read_text).pack()

window.mainloop()
```

■ ***Note:*** *"1.0" means line 1, character 0 (the very beginning). tk.END means the very end.*

2.5 Checkbutton & Radiobutton

Use **Checkbutton** for yes/no choices, and **Radiobutton** when only one option from a group can be selected.

```
import tkinter as tk

window = tk.Tk()
window.title("Check & Radio")
```

```
# --- Checkbutton ---
var_check = tk.BooleanVar()   # stores True/False
chk = tk.Checkbutton(window, text="I agree to terms", variable=var_check)
chk.pack(anchor="w", padx=20)

# --- Radiobutton (group shares same variable) ---
var_radio = tk.StringVar(value="Python")   # default selection

for lang in ["Python", "Java", "C++"]:
    rb = tk.Radiobutton(window, text=lang,
                        variable=var_radio, value=lang)
    rb.pack(anchor="w", padx=20)

def show():
    print("Agreed:", var_check.get())
    print("Language:", var_radio.get())

tk.Button(window, text="Show Selections", command=show).pack(pady=10)
window.mainloop()
```

3 Layout Managers

Layout managers decide **where** widgets appear inside a window. Tkinter has three: **pack**, **grid**, and **place**.

3.1 pack() — Stack Widgets

pack() stacks widgets one after another. It is the simplest layout manager.

```
import tkinter as tk
window = tk.Tk()

tk.Label(window, text="Top",    bg="lightblue").pack(fill="x")
tk.Label(window, text="Middle", bg="lightyellow").pack(fill="x")
tk.Label(window, text="Bottom", bg="lightgreen").pack(fill="x")

window.mainloop()
```

Option	Values	Meaning
side	TOP, BOTTOM, LEFT, RIGHT	Which side to attach to
fill	x, y, both, none	Stretch to fill space
expand	True / False	Let widget grow to fill window
padx / pady	number (pixels)	Outer spacing around widget
ipadx/ipady	number (pixels)	Inner padding inside widget
anchor	n,s,e,w,center,…	Alignment when not filling

3.2 grid() — Row and Column Layout

grid() places widgets in a table-like structure with rows and columns. This is ideal for forms.

```
import tkinter as tk
window = tk.Tk()
window.title("Grid Layout")

# Labels (column 0)
tk.Label(window, text="Name:").grid(row=0, column=0, sticky="e", padx=5, pady=5)
tk.Label(window, text="Email:").grid(row=1, column=0, sticky="e", padx=5, pady=5)
tk.Label(window, text="Phone:").grid(row=2, column=0, sticky="e", padx=5, pady=5)

# Entry boxes (column 1)
```

```
name_entry  = tk.Entry(window, width=25)
email_entry = tk.Entry(window, width=25)
phone_entry = tk.Entry(window, width=25)

name_entry.grid(row=0, column=1, padx=5, pady=5)
email_entry.grid(row=1, column=1, padx=5, pady=5)
phone_entry.grid(row=2, column=1, padx=5, pady=5)

# Button spanning 2 columns
tk.Button(window, text="Submit", width=15).grid(
    row=3, column=0, columnspan=2, pady=10)

window.mainloop()
```

sticky controls alignment: 'e' = right-align, 'w' = left-align, 'nsew' = stretch in all directions.

3.3 place() — Exact Pixel Positioning

place() lets you set exact x/y coordinates. Good for specific layouts but hard to maintain.

```
btn = tk.Button(window, text="Fixed Position")
btn.place(x=100, y=150)    # 100px from left, 150px from top

# Or use relative positioning (0.0 to 1.0):
btn2 = tk.Button(window, text="Relative")
btn2.place(relx=0.5, rely=0.5, anchor="center")  # exact center
```

3.4 Which Layout to Use?

Situation	Best layout
Quick prototype / simple stacking	pack()
Forms with labels + inputs	grid()
Precise positioning	place()
■ Never mix pack() and grid() in the same Frame!	

4 Event Handling

An **event** is anything the user does — a mouse click, pressing a key, moving the cursor. Tkinter lets you 'listen' for events and run code when they happen.

4.1 command= (Simplest Way)

You already saw this: pass a function to the **command** option of a Button.

```
def on_click():
    label.config(text="Button clicked!")

btn = tk.Button(window, text="Click", command=on_click)
```

4.2 bind() — Attach Any Event

bind() lets you attach events to **any widget**, not just buttons.

```
import tkinter as tk

window = tk.Tk()
window.title("Events Demo")

label = tk.Label(window, text="Try events!", font=("Arial", 14))
label.pack(pady=30)

# Mouse click on label
def on_left_click(event):
    label.config(text=f"Left-clicked at ({event.x}, {event.y})")

def on_right_click(event):
    label.config(text="Right-clicked!")

def on_key(event):
    label.config(text=f"Key pressed: {event.keysym}")

label.bind("<Button-1>",  on_left_click)   # left mouse button
label.bind("<Button-3>",  on_right_click)  # right mouse button
window.bind("<Key>",       on_key)          # any keyboard key

window.mainloop()
```

4.3 Common Event Strings

Event String	When it fires
`<Button-1>`	Left mouse click
`<Button-2>`	Middle mouse click
`<Button-3>`	Right mouse click
`<Double-Button-1>`	Double left click
`<Enter>`	Mouse cursor enters widget
`<Leave>`	Mouse cursor leaves widget
`<Key>`	Any key pressed
`<Return>`	Enter/Return key pressed
`<BackSpace>`	Backspace key pressed
`<FocusIn>`	Widget receives focus
`<FocusOut>`	Widget loses focus

4.4 Lambda for Quick Inline Functions

If you want to pass an argument to a command function, use **lambda**:

```
def show_message(msg):
    print(msg)

# Without lambda this would call show_message immediately at startup:
# WRONG:  command=show_message("Hello")
# RIGHT:
btn = tk.Button(window, text="Hi",
                command=lambda: show_message("Hello from button!"))
```

5 More Useful Widgets

5.1 Listbox — Scrollable List

```
import tkinter as tk

window = tk.Tk()
fruits = ["Apple", "Banana", "Cherry", "Mango", "Orange"]

listbox = tk.Listbox(window, height=5, selectmode=tk.SINGLE)
for fruit in fruits:
    listbox.insert(tk.END, fruit)        # add items to end
listbox.pack(padx=10, pady=10)

def show_selected():
    idx = listbox.curselection()         # returns a tuple of indices
    if idx:
        print(listbox.get(idx[0]))       # get text at that index

tk.Button(window, text="Show Selected", command=show_selected).pack()
window.mainloop()
```

5.2 Combobox — Dropdown Menu

Combobox is in the **ttk** (themed Tkinter) sub-module.

```
import tkinter as tk
from tkinter import ttk

window = tk.Tk()

combo = ttk.Combobox(window, values=["Python", "Java", "C++", "JavaScript"])
combo.set("Python")    # default value
combo.pack(pady=10)

def on_select(event):
    print("Selected:", combo.get())

combo.bind("<<ComboboxSelected>>", on_select)
window.mainloop()
```

5.3 Spinbox — Numeric Input

```
spinbox = tk.Spinbox(window, from_=1, to=100, width=5)
spinbox.pack()
print(spinbox.get())   # current value
```

5.4 Scale — Slider Widget

```
var = tk.IntVar()
scale = tk.Scale(window, from_=0, to=100,
                 orient=tk.HORIZONTAL,   # or VERTICAL
                 variable=var,
                 label="Volume")
scale.pack()
print(var.get())   # current slider value
```

5.5 MessageBox — Pop-up Dialogs

messagebox lets you show pop-up alerts, confirmations, and errors.

```
from tkinter import messagebox

# Information pop-up
messagebox.showinfo("Title", "This is an info message")

# Warning
messagebox.showwarning("Warning", "Something might go wrong!")

# Error
messagebox.showerror("Error", "Something went wrong!")

# Yes/No question — returns True or False
answer = messagebox.askyesno("Confirm", "Do you want to delete?")
if answer:
    print("User said YES")
else:
    print("User said NO")
```

6 Frames & Organisation

As your app grows, you need to organise widgets into groups. Frames are containers that hold other widgets.

6.1 Frame — Basic Container

```
import tkinter as tk

window = tk.Tk()
window.title("Frame Demo")

# Top frame (blue background)
top_frame = tk.Frame(window, bg="lightblue", pady=10)
top_frame.pack(fill="x")

tk.Label(top_frame, text="Header Section",
         bg="lightblue", font=("Arial",14,"bold")).pack()

# Bottom frame (yellow background)
bottom_frame = tk.Frame(window, bg="lightyellow", pady=10)
bottom_frame.pack(fill="both", expand=True)

tk.Button(bottom_frame, text="Button 1").pack(side="left", padx=10)
tk.Button(bottom_frame, text="Button 2").pack(side="left", padx=10)
tk.Button(bottom_frame, text="Button 3").pack(side="left", padx=10)

window.mainloop()
```

6.2 LabelFrame — Frame with a Title

```
lf = tk.LabelFrame(window, text=" Personal Details ",
                   font=("Arial",10,"bold"), padx=10, pady=10)
lf.pack(padx=20, pady=20, fill="x")

tk.Label(lf, text="Name:").grid(row=0, column=0, sticky="e")
tk.Entry(lf, width=20).grid(row=0, column=1, padx=5)
tk.Label(lf, text="Age:").grid(row=1, column=0, sticky="e")
tk.Entry(lf, width=20).grid(row=1, column=1, padx=5, pady=5)
```

6.3 Notebook — Tabbed Interface

ttk.Notebook creates a tabbed layout — perfect for organising complex apps.

```
import tkinter as tk
from tkinter import ttk

window = tk.Tk()
window.title("Notebook Tabs")

notebook = ttk.Notebook(window)
notebook.pack(fill="both", expand=True, padx=10, pady=10)

# Create frames for each tab
tab1 = tk.Frame(notebook, bg="white")
tab2 = tk.Frame(notebook, bg="lightyellow")
tab3 = tk.Frame(notebook, bg="lightblue")

# Add tabs to notebook
notebook.add(tab1, text=" ■ Home ")
notebook.add(tab2, text=" ■ Profile ")
notebook.add(tab3, text=" ■ Settings ")

# Add content to tab1
tk.Label(tab1, text="Welcome to Home Tab!",
         font=("Arial",14), bg="white").pack(pady=30)

window.mainloop()
```

7 Forms & Input Validation

Most real apps collect data from users. Let's build a proper form with validation to ensure the user enters correct data before we process it.

7.1 Complete Registration Form with Validation

```
import tkinter as tk
from tkinter import messagebox

window = tk.Tk()
window.title("Registration Form")
window.geometry("400x350")
window.resizable(False, False)

# ■■ Title ■■■
```

```
# ■■ Validation logic ■■■
```

```
    tk.Button(window, text="Register", command=validate_and_submit,
              bg="#1565c0", fg="white", font=("Arial",11),
              width=15).pack(pady=20)

    window.mainloop()
```

> ■ ***Tip:*** *Always validate before processing data. Never assume the user typed the right thing.*

8 SQLite & Python's sqlite3 Module

SQLite is a lightweight, file-based database. It stores data in a single **.db** file on your computer — no server needed. Python includes the **sqlite3** module built-in, so you can use a database without installing anything extra.

8.1 How a Database Works (Basics)

Think of a database as an Excel spreadsheet:

Database concept	Real-world analogy
Database (.db file)	The whole Excel workbook
Table	One sheet in the workbook
Column	A column header (Name, Age, …)
Row / Record	One row of data
Primary Key	A unique ID for each row (like a roll number)

8.2 Connecting and Creating a Table

```
import sqlite3

# Step 1: Connect to a database file (creates it if it doesn't exist)
conn = sqlite3.connect("school.db")

# Step 2: Create a cursor (a tool to run SQL commands)
cursor = conn.cursor()

# Step 3: Create a table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        id      INTEGER PRIMARY KEY AUTOINCREMENT,
        name    TEXT    NOT NULL,
        age     INTEGER,
        grade   TEXT,
        email   TEXT
    )
''')
```

```
# Step 4: Commit (save) the changes
conn.commit()

# Step 5: Close the connection
conn.close()

print("Database and table created successfully!")
```

■ ***Note:*** *Always call conn.commit() after INSERT/UPDATE/DELETE. Without it, changes are not saved.*

8.3 The 4 CRUD Operations in SQL

Operation	SQL Command	What it does
Create	INSERT INTO ...	Add a new row
Read	SELECT * FROM ...	Fetch rows
Update	UPDATE ... SET ...	Modify a row
Delete	DELETE FROM ... WHERE ...	Remove a row

8.4 SQL CRUD Examples (Pure Python, No GUI Yet)

```
import sqlite3

conn   = sqlite3.connect("school.db")
cursor = conn.cursor()

# ■■ CREATE (Insert) ■■■
```

```
cursor.execute("SELECT * FROM students WHERE id = ?", (1,))
student = cursor.fetchone()
print(student)

# ■■ UPDATE ■■■
```

9 CRUD Operations with Tkinter GUI

Now we combine Tkinter and SQLite. We'll use **ttk.Treeview** to display database records in a table format inside our window.

9.1 Treeview — Displaying Table Data

```
import tkinter as tk
from tkinter import ttk

window = tk.Tk()
window.title("Treeview Demo")

# Define columns
columns = ("ID", "Name", "Age", "Grade")
tree = ttk.Treeview(window, columns=columns, show="headings", height=8)

# Set column headings and widths
tree.heading("ID",    text="ID")
tree.heading("Name",  text="Name")
tree.heading("Age",   text="Age")
tree.heading("Grade", text="Grade")

tree.column("ID",    width=50,  anchor="center")
tree.column("Name",  width=150, anchor="w")
tree.column("Age",   width=60,  anchor="center")
tree.column("Grade", width=60,  anchor="center")

tree.pack(padx=10, pady=10)

# Add a scrollbar
scrollbar = ttk.Scrollbar(window, orient="vertical", command=tree.yview)
tree.configure(yscrollcommand=scrollbar.set)
scrollbar.pack(side="right", fill="y")

# Insert sample data
sample_data = [
    (1, "Alice",   20, "A"),
```

```
    (2, "Bob",     22, "B"),
    (3, "Charlie", 21, "A+"),
]
for row in sample_data:
    tree.insert("", tk.END, values=row)

window.mainloop()
```

9.2 Reading Selected Row from Treeview

```
def on_row_select(event):
    selected = tree.selection()           # returns selected item IDs
    if selected:
        item = tree.item(selected[0])     # get item data
        values = item["values"]           # list: [id, name, age, grade]
        print("Selected:", values)
        # Fill form fields:
        name_entry.delete(0, tk.END)
        name_entry.insert(0, values[1])

tree.bind("<<TreeviewSelect>>", on_row_select)
```

10 Full Project: Student Management System

This is a **complete, working application** that combines everything you have learned. It manages student records with full Create, Read, Update, and Delete operations, all connected to an SQLite database.

Features of this app:

- Add new students
- View all students in a table
- Select a row to auto-fill the form
- Update student details
- Delete a student with confirmation
- Search by name
- Clear form button
- Input validation

10.1 Complete Source Code

■ ***Note:*** *Copy this entire code into a file called student_app.py and run it with: python student_app.py*

```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
#  DATABASE SETUP
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def init_db():
    """Create the database and students table if they don't exist."""
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS students (
            id    INTEGER PRIMARY KEY AUTOINCREMENT,
            name  TEXT    NOT NULL,
            age   INTEGER,
            grade TEXT,
```

```
            email TEXT
        )
    """)
    conn.commit()
    conn.close()

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
#  DATABASE OPERATIONS
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def add_student(name, age, grade, email):
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO students (name, age, grade, email) VALUES (?,?,?,?)",
        (name, age, grade, email)
    )
    conn.commit()
    conn.close()

def get_all_students():
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM students ORDER BY name")
    rows = cursor.fetchall()
    conn.close()
    return rows

def search_students(keyword):
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()
    cursor.execute(
        "SELECT * FROM students WHERE name LIKE ?",
        (f"%{keyword}%",)
    )
    rows = cursor.fetchall()
```

```
    conn.close()
    return rows

def update_student(sid, name, age, grade, email):
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()
    cursor.execute("""
        UPDATE students
        SET name=?, age=?, grade=?, email=?
        WHERE id=?
    """, (name, age, grade, email, sid))
    conn.commit()
    conn.close()

def delete_student(sid):
    conn = sqlite3.connect("students.db")
    cursor = conn.cursor()
    cursor.execute("DELETE FROM students WHERE id=?", (sid,))
    conn.commit()
    conn.close()
```

```
# ═══════════════════════════════════════════════════════════════
#  MAIN APPLICATION CLASS
# ═══════════════════════════════════════════════════════════════
class StudentApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Student Management System")
        self.root.geometry("820x600")
        self.root.resizable(True, True)
        self.selected_id = None          # stores the ID of selected row
        self.build_ui()
        self.refresh_table()

    # ── UI CONSTRUCTION ──────────────────────────────────────
    def build_ui(self):
```

```
        # Title bar
        header = tk.Frame(self.root, bg="#1a237e", height=55)
        header.pack(fill="x")
        tk.Label(header, text="  Student Management System",
                 font=("Arial", 16, "bold"),
                 fg="white", bg="#1a237e").pack(side="left", pady=12)

        # Main body: left (form) + right (table)
        body = tk.Frame(self.root)
        body.pack(fill="both", expand=True, padx=10, pady=10)

        self.build_form(body)
        self.build_table(body)

    def build_form(self, parent):
        """Left panel: input form."""
        form_frame = tk.LabelFrame(parent, text=" Student Details ",
                                   font=("Arial",10,"bold"),
                                   padx=12, pady=12, width=280)
        form_frame.pack(side="left", fill="y", padx=(0,10))
        form_frame.pack_propagate(False)

        labels = ["Name *", "Age *", "Grade", "Email"]
        self.entries = {}

        for i, lbl in enumerate(labels):
            tk.Label(form_frame, text=lbl).grid(
                row=i, column=0, sticky="e", pady=7)
            e = tk.Entry(form_frame, width=22)
            e.grid(row=i, column=1, padx=8, pady=7)
            self.entries[lbl] = e

        # Search row
        tk.Label(form_frame, text="Search:").grid(
            row=4, column=0, sticky="e", pady=(15,5))
        self.search_var = tk.StringVar()
        self.search_var.trace("w", lambda *a: self.do_search())
```

```
        tk.Entry(form_frame, textvariable=self.search_var,
                 width=22).grid(row=4, column=1, padx=8)

        # Buttons
        btn_cfg = dict(font=("Arial",10), width=12)
        tk.Button(form_frame, text="■ Add",
                  command=self.add, bg="#2e7d32", fg="white",
                  **btn_cfg).grid(row=5, column=0, columnspan=2, pady=(20,4))
        tk.Button(form_frame, text="═ Update",
                  command=self.update, bg="#1565c0", fg="white",
                  **btn_cfg).grid(row=6, column=0, columnspan=2, pady=4)
        tk.Button(form_frame, text="■ Delete",
                  command=self.delete, bg="#c62828", fg="white",
                  **btn_cfg).grid(row=7, column=0, columnspan=2, pady=4)
        tk.Button(form_frame, text="■ Clear",
                  command=self.clear_form, bg="#546e7a", fg="white",
                  **btn_cfg).grid(row=8, column=0, columnspan=2, pady=4)
```

```
    def build_table(self, parent):
        """Right panel: Treeview table."""
        table_frame = tk.Frame(parent)
        table_frame.pack(side="right", fill="both", expand=True)

        cols = ("ID", "Name", "Age", "Grade", "Email")
        self.tree = ttk.Treeview(table_frame, columns=cols,
                                 show="headings", height=20)

        widths = {"ID":45, "Name":160, "Age":55, "Grade":65, "Email":180}
        for col in cols:
            self.tree.heading(col, text=col,
                command=lambda c=col: self.sort_column(c))
            self.tree.column(col, width=widths[col], anchor="center")

        self.tree.column("Name",  anchor="w")
        self.tree.column("Email", anchor="w")

        # Scrollbars
```

```
        vsb = ttk.Scrollbar(table_frame, orient="vertical",
                            command=self.tree.yview)
        hsb = ttk.Scrollbar(table_frame, orient="horizontal",
                            command=self.tree.xview)
        self.tree.configure(yscrollcommand=vsb.set,
                            xscrollcommand=hsb.set)

        self.tree.pack(side="left", fill="both", expand=True)
        vsb.pack(side="right",  fill="y")
        hsb.pack(side="bottom", fill="x")

        # Alternating row colours
        self.tree.tag_configure("even", background="#e3f2fd")
        self.tree.tag_configure("odd",  background="white")

        self.tree.bind("<<TreeviewSelect>>", self.on_select)
```

```
    # ■■ CRUD METHODS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    def validate(self):
        """Returns (name, age, grade, email) or None on error."""
        name  = self.entries["Name *"].get().strip()
        age   = self.entries["Age *"].get().strip()
        grade = self.entries["Grade"].get().strip()
        email = self.entries["Email"].get().strip()

        if not name:
            messagebox.showerror("Error", "Name is required!"); return None
        if not age.isdigit():
            messagebox.showerror("Error", "Age must be a number!"); return None
        return name, int(age), grade, email

    def add(self):
        data = self.validate()
        if data:
            add_student(*data)
            self.refresh_table()
            self.clear_form()
```

```
            messagebox.showinfo("Success", "Student added successfully!")

    def update(self):
        if not self.selected_id:
            messagebox.showwarning("Warning", "Please select a student first!")
            return
        data = self.validate()
        if data:
            update_student(self.selected_id, *data)
            self.refresh_table()
            messagebox.showinfo("Success", "Student updated!")

    def delete(self):
        if not self.selected_id:
            messagebox.showwarning("Warning", "Please select a student first!")
            return
        name = self.entries["Name *"].get()
        ok = messagebox.askyesno("Confirm Delete",
                                 f"Delete student '{name}'?\nThis cannot be undone.")
        if ok:
            delete_student(self.selected_id)
            self.refresh_table()
            self.clear_form()
            messagebox.showinfo("Deleted", "Student deleted successfully!")

    # ■■ HELPER METHODS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    def refresh_table(self, rows=None):
        """Clear and reload the Treeview."""
        for item in self.tree.get_children():
            self.tree.delete(item)
        if rows is None:
            rows = get_all_students()
        for i, row in enumerate(rows):
            tag = "even" if i % 2 == 0 else "odd"
            self.tree.insert("", tk.END, values=row, tags=(tag,))
```

```
    def on_select(self, event):
        """Fill form when a row is clicked."""
        sel = self.tree.selection()
        if not sel: return
        vals = self.tree.item(sel[0])["values"]
        self.selected_id = vals[0]
        fields = ["Name *", "Age *", "Grade", "Email"]
        for field, val in zip(fields, vals[1:]):
            self.entries[field].delete(0, tk.END)
            self.entries[field].insert(0, val)

    def clear_form(self):
        for e in self.entries.values():
            e.delete(0, tk.END)
        self.selected_id = None
        self.search_var.set("")

    def do_search(self):
        kw = self.search_var.get().strip()
        if kw:
            self.refresh_table(search_students(kw))
        else:
            self.refresh_table()

    def sort_column(self, col):
        """Sort Treeview by column on heading click."""
        items = [(self.tree.set(k, col), k)
                 for k in self.tree.get_children("")]
        items.sort()
        for idx, (_, k) in enumerate(items):
            self.tree.move(k, "", idx)
```

```
# ═══════════════════════════════════════════════════════════════
#  ENTRY POINT
# ═══════════════════════════════════════════════════════════════
```

```
if __name__ == "__main__":
    init_db()                          # set up database first
    root = tk.Tk()
    app  = StudentApp(root)
    root.mainloop()
```

> ■ ***Tip:*** *Save this as student_app.py and run: python student_app.py — A students.db file will be created automatically in the same folder.*

11 Tips, Best Practices & Next Steps

11.1 Common Beginner Mistakes

■ Mistake	■ Fix
Calling function with () in command=	Write command=my_func, not command=my_func()
Mixing pack() and grid() in same frame	Use only one layout manager per container
Forgetting conn.commit()	Always call commit() after INSERT/UPDATE/DELETE
Not using ? in SQL queries	Always use ? placeholders — never f-strings in SQL
Forgetting mainloop()	Last line of your app must always be window.mainloop()
Storing the widget but forgetting .pack()/.grid()	Every widget needs a layout call to become visible

11.2 Best Practices

- **Use a class for your app.** Put all your widgets and methods inside one class. This keeps code clean and organised (see Chapter 10).
- **Separate database code from GUI code.** Keep SQL functions in their own section or file.
- **Always validate user input** before saving to the database.
- **Use ttk widgets** (ttk.Button, ttk.Entry, etc.) for a more modern look.
- **Name your variables clearly** — name_entry is better than e1.
- **Handle exceptions** around database calls using try/except to avoid crashes.

11.3 Packaging Your App

To share your app as a standalone .exe (Windows) or binary (Mac/Linux) without Python installed:

```
# Install PyInstaller
pip install pyinstaller

# Package your app (creates a dist/ folder)
pyinstaller --onefile --windowed student_app.py

# The executable is in:  dist/student_app.exe  (Windows)
#                        dist/student_app       (Mac/Linux)
```

11.4 What to Learn Next

Topic	Why learn it?

ttk Themes (ttkthemes)	Give your app a modern, beautiful look
tkinter.filedialog	Let users open/save files
PIL / Pillow	Display and manipulate images in your GUI
Matplotlib in Tkinter	Embed charts and graphs in your app
Threading	Run tasks in background without freezing GUI
PostgreSQL / MySQL	Scale beyond SQLite for bigger projects
PyQt5 / PySide6	More powerful GUI library for complex apps

You've completed the Python Tkinter Beginner's Guide!

Keep building. Keep learning. ■